

Project Report

Project Name: Rescue Pawtrol

Reference ID: XGYOI2D7

1. Executive Summary

Rescue Pawtrol is a browser-based arcade game built with Phaser 3 in which players work through three linked mini-games to save stray cats: exploring a city map to gather and shelter cats while dodging hazards, defending an injured cat from incoming projectiles with a moving shield, and triaging rescued cats by matching them to the correct treatment station. All three levels are playable end-to-end, with a growing list of bugs found and fixed during testing. A live build is hosted at rescue-pawtrol.netlify.app.

2. Project Overview

2a. Why you're building what you're building

I am currently in an organization called Project Pawradise, and from what I've seen back home, people treat strays as filthy, feral, and dangerous. But being in the organization and being a gamer as well, it motivated me to create a game to advocate for stray awareness. Searching for animals, protecting them from danger, and providing them with the right care are each modeled as distinct gameplay mechanics, so the game builds empathy for what real-world rescue work entails.

2b. How it relates to the theme

Since the theme is World Cat Domination Day, the game revolves around cats. Every level centers cats and their rescue as the core mechanic: finding and carrying cats to safety, shielding an injured cat from harm, and treating rescued cats with the right supplies, making the connection to the theme direct rather than incidental.

2c. Target Audience

This game is designed for casual players, arcade web game enthusiasts, and animal lovers who appreciate fast-paced, high-tension, top-down retro games with deep procedural gameplay. It's also for all ages.

3. Key Features

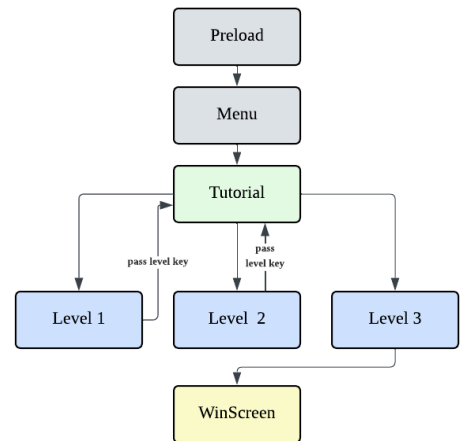
- ☐ Three distinct mini-games in one game: an open-world rescue level with a shop-style upgrade panel (Level 1), a wave-based shield-defense survival level (Level 2), and a click-to-match cat triage level (Level 3).
- ☐ Reusable tutorial scene that dynamically renders different instructions depending on which level is about to start.
- ☐ Escalating difficulty: Level 2 ramps enemy speed and spawn rate across 3 waves, and Level 3's spawn delay shortens over time as the round progresses.
- ☐ In-run upgrade systems: a currency-based shop in Level 1 (speed boost, carry-two-cats, camera zoom, shield, slow enemies, reveal) and a pick-one-of-three upgrade screen between waves in Level 2
- ☐ Full audio integration, with looping background music per level and dedicated sound effects for hits, wins, losses, and correct/incorrect matches, including distinct win/lose stingers in each level.

4. Technology Stack

Layer	Technology
Frontend / Game Engine	Phaser 3 (JavaScript, Arcade Physics)
Rendering	HTML5 Canvas / WebGL (via Phaser)
Audio	Phaser Sound Manager (MP3 assets)
Assets	Custom PNG sprites/spritesheets and MP3 audio files

5. Technical Architecture

The game runs on Phaser 3 with a single Arcade Physics world (main.js) that registers seven scenes in sequence. Preload loads all sprite sheets, images, and audio, then hands off to Menu. Tutorial is a reusable scene that renders different instructions depending on the level key passed to it via scene data. Level1 is a top-down explore-and-rescue level with a scrolling camera, a shop-style upgrade panel, roving enemies, and periodic hazard timers, with a zoomed-in starting camera that can be reset to normal via an upgrade. Level2 is a fixed-camera defense level driven by a 3-wave timer system, where incoming projectiles are spawned from the screen edges and must be intercepted by a mouse-aimed shield before they reach a cat at the center. Level3 is a click-based triage level where waiting cats display a need icon that must be matched to one of three treatment stations before a countdown reaches zero. WinScreen reads cat-rescued and cat-treated counts out of Phaser's scene registry. Values written by Level1 and Level3 as the player progresses -- to show a final recap with a button back to the Menu. Each scene owns its own state (score, waves, timers) and calls `this.scene.start()` to move to the next scene, so control flow is linear: Preload -> Menu -> Tutorial -> Level1 -> Tutorial -> Level2 -> Tutorial -> Level3 -> WinScreen.



6. Testing Matrix

Feature / Flow	Steps	Expected Result	Actual Result	Pass / Fail
Cat Collection	Player runs over cat	Cat is removed from the static group; visual asset appends to trailing queue.	As expected, ghost hitboxes are completely removed.	PASS
Cat drop-off	Carry a picked-up cat to the drop-off zone near the shelter	Cat is banked, and Love currency increases	Fixed -- Love is now actually deducted per upgrade cost instead of always being free	PASS
Level 2 shield block	Move the mouse so the shield intercepts an incoming projectile	Projectile is destroyed, and the shield flashes white	Fixed -- rock/virus hitboxes now derive from each sprite's real texture size instead of an assumed 256x256	PASS
Level 3 cat-station matching	Click a waiting cat, then click the station matching its need icon	Cat is treated, saved count increases, and score text updates	Fixed -- win/lose now play the correct sound and route to the correct screen	PASS

8. Future Improvements

- ☐ Custom Sprites and Audios
- ☐ Advanced Leaderboard Systems
- ☐ Phone Compatibility

9. Tools You Used

- ☐ Phaser 3
- ☐ Claude
- ☐ Aseprite
- ☐ VS Code

11. Learnings & Takeaways

Maybe two heads are better than one, but not all the time. Being the first hackathon I ever joined, I was nervous, but good thing I joined with fear because it is a good place to start.

12. Acknowledgments

I'd like to thank the art community for making free sprites for unemployed people like me. I would also like to thank my teachers for lighting the spark in me.

Submission Checklist

- ☐ Video demo (HD or at least 720p)
- ☐ README.md (prerequisites, run instructions, configuration)
- ☐ Project report (this document, in documentation/)
- ☐ Source code in src/
- ☐ No unrelated files, executables, auto-generated code, or package folders (e.g. node_modules/, build/, dist/, vendor/)